ИТМО

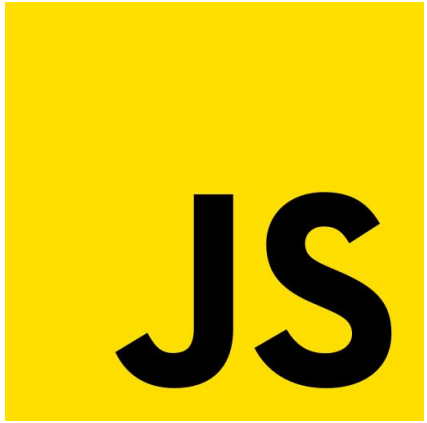
HTML, CSS и JavaScript: базовый frontend без фреймворков

Роли HTML, CSS, JS

ИТМО







HTML

ИТМО

Что это

HTML (HyperText Markup Language) — это язык разметки для создания структуры и текста веб-страниц.

И как это

Определение

```
<имя></имя>
или
<имя/>
или
<имя>
```

Атрибут

```
<.. атрибут="знач">
```

Структура HTML-документа

ИТМО

```
<!doctype html>
<html lang=”ru”>
     <head>
          <meta charset="UTF-8">
          <meta name="viewport" ...>
          <title>Title!</title>
     </head>
     <body>
          …
          <link rel="stylesheet"
href="style.css">
          <script src="script.js"
defer></script>
     </body>
</html>
```

← указание типа текущего документа
← корневой элемент страницы
← атрибут метаданных
← указание используемой кодировки
← информация об отображении для мобильных устройств
← заголовок страницы

← контент страницы

← подключение файлов CSS

← подключение файлов JavaScript

Структура HTML-документа

```
<header>
    <nav>
        <a href="/">Главная</a>
    </nav>
</header>
```

Дерево документа

Порядок закрытия важен!

Основные атрибуты:

- id (идентификатор)
- class (класс)
- href/src (ссылки/внешние источники)
- alt (текстовое описание изображения)
- data-* (спец. атрибуты)

Семантический HTML



Семантический код - это подход к построению структуры страницы, при котором элемент используется согласно его предназначению и определению

- Повышение доступности
- Упрощение разработки
- Эффективнее веб-скрапинг и SEO

Семантический HTML

ИТМО

Очень плохо, не делайте так	Очень хорошо, делайте так

`<div class="header">...`
`<div class="nav">...`
`<div class="main">...`
`<div class="sidebar">...`
`<div class="footer">...`
	`<header>...`
	`<nav>...`
	`<main>...`
	`<aside>...`
	`<footer>...`

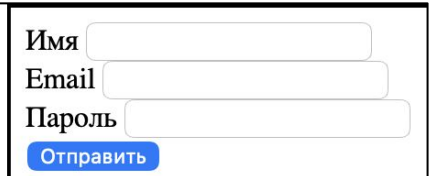
Основные семантические элементы:

- header
- nav
- main
- article
- section
- aside
- footer
- figure
- h1-h6

HTML-формы

ИТМО

```html
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial
        -scale=1.0">
    <title>Простая форма</title>
</head>
<body>
    <form>
        <div>
            <label for="name">Имя</label>
            <input type="text" id="name" name="name">
        </div>
        <div>
            <label for="email">Email</label>
            <input type="email" id="email" name="email">
        </div>
        <div>
            <label for="password">Пароль</label>
            <input type="password" id="password" name="password">
        </div>
        <div>
            <button type="submit">Отправить</button>
        </div>
    </form>
</body>
</html>
```



HTML-формы

ИТМО

Другие основные элементы:

- <textarea> — для многострочного текста
- <select> с вложенными <option> — выпадающий список
- <button> — кнопка. Имеет несколько типов, определяемых атрибутом type
- <label> — подпись к полю. Связать можно двумя способами: вложить <input> в <label> или использовать атрибуты id у поля и for у <label>
- <fieldset> и <legend> — группировка полей

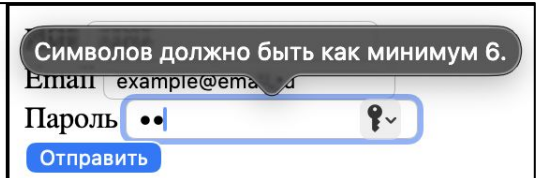
Ключевые атрибуты:

- name — это имя ключа, под которым данные уйдут на сервер
- value — значение по умолчанию
- type - тип поля ввода. Используется для определения применяемых валидаторов и представления поля. Основные - text, checkbox, textarea, button, radio
- required — говорит, что поле обязательно для заполнения
- placeholder — подсказка внутри поля, исчезает при вводе
- disabled — поле неактивно, его данные не отправляются
- readonly — только для чтения, но данные отправляются

HTML-формы

ИТМО

```
1  <!DOCTYPE html>
2  <html lang="ru">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial
           -scale=1.0">
6      <title>Форма с валидацией</title>
7  </head>
8  <body>
9      <form action="/submit" method="post">
10         <div>
11             <label for="name">Имя</label>
12             <input type="text" id="name" name="name" required>
13         </div>
14         <div>
15             <label for="email">Email</label>
16             <input type="email" id="email" name="email" required>
17         </div>
18         <div>
19             <label for="password">Пароль</label>
20             <input type="password" id="password" name="password"
                   required minlength="6">
21         </div>
22         <div>
23             <button type="submit">Отправить</button>
24         </div>
25     </form>
26 </body>
27 </html>
```



Встроенная браузерная валидация

CSS

ИТМО

Что это, как это

CSS (Cascade Style Sheets) — это язык описания внешнего вида веб-страниц.

```
имя_селектор {
  имя_свойства: значение;
}
```

Селектор - шаблон, указывающий, к какому элементу применить стили

Куда это (приемлемые варианты)

```
<link rel="stylesheet"
href="style.css">
```

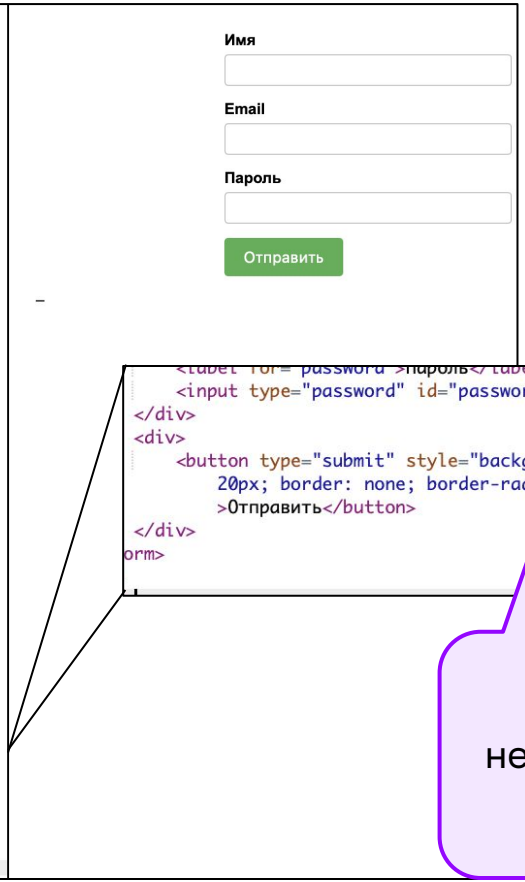
или

```
<head>
     <style>...</style>
</head>
```

CSS

ИТМО

```html
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Форма со стилями</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            padding: 20px;
        }
        form {
            max-width: 300px;
            margin: 0 auto;
        }
        div {
            margin-bottom: 15px;
        }
        label {
            display: block;
            margin-bottom: 5px;
            font-weight: bold;
        }
        input {
            width: 100%;
            padding: 8px;
            border: 1px solid #ccc;
            border-radius: 4px;
            box-sizing: border-box;
        }
        input:focus {
            border-color: #4CAF50;
            outline: none;
        }
    </style>
</head>
<body>
    <form action="/submit" method="post">
        <div>
            <label for="name">Имя</label>
            <input type="text" id="name" name="name" required>
        </div>
        <div>
            <label for="email">Email</label>
            <input type="email" id="email" name="email" required>
        </div>
        <div>
            <label for="password">Пароль</label>
            <input type="password" id="password" name="password" required minlength="6">
        </div>
        <div>
            <button type="submit" style="background-color: #4CAF50; color: white; padding: 10px
                20px; border: none; border-radius: 4px; cursor: pointer; font-size: 16px;"
                >Отправить</button>
        </div>
    </form>
</body>
</html>
```




```
    <input type="password" id="password" name="password" required minlength="6">
</div>
<div>
    <button type="submit" style="background-color: #4CAF50; color: white; padding: 10px
        20px; border: none; border-radius: 4px; cursor: pointer; font-size: 16px;"
        >Отправить</button>
</div>
```

Очень нежелательный вариант

Основные понятия CSS

ИТМО

Селекторы

Типы:

- По элементу: p, h1
- По классу: .my-class
- По идентификатору: #my-id
- По атрибуту: input[type="email"]

Псевдоклассы

Описывают состояние элемента

Состояние пользователя и системы:

- :hover
- :focus
- :active
- :checked
- :disabled

Положение в структуре документа:

- :first-child
- :last-child
- :nth-child(n)

Логические псевдоклассы:

- :not(.active)

Псевдоэлементы

Описывают стилизацию определенной части элемента

Популярные:

- ::before
- ::after
- ::placeholder
- ::selection
- ::first-letter

Каскад CSS

ИТМО

Факторы анализа приоритетности стилей:

1. Источник стилей
   Важнее всего стили разработчика, потом браузера, потом пользователя (порядок меняется при использовании !important)
2. Специфичность
   Набор компонентов, определяется по весу:
   a. Универсальный `*` = 0
   b. Элемент и псевдоэлемент (`h1`) = 1
   c. Класс, псевдокласс, атрибут (`.title`) = 10
   d. Идентификатор (`#header`) = 100
   e. Инлайн-стиль = 1000
3. Порядок
   При равенстве всего побеждает то правило, что объявлено последним

При сложном определении веса складываются

Общий порядок применения стилей:

1. Стили во время CSS-переходов (свойство transition)
2. Браузерные стили с !important
3. Пользовательские стили с !important
4. Авторские стили с !important (включая inline)
5. Стили во время анимаций (свойство animation)
6. Обычные авторские стили
   a. inline
   b. id
   c. классы/атрибуты/псевдоклассы
   d. теги/псевдоэлементы
7. Обычные пользовательские стили
8. Обычные браузерные стили

CSS-переменные

ИТМО

```
:root {
    --main-color: #3498db;
    --secondary-color: #341234;
    --prim-height: 20px;
    --secondary-height: 25px;
}
a {
    color: var(--main-color, var(--secondary-color, pink));
    height: var(--prim-height, --secondary-height, 30px);
}
```

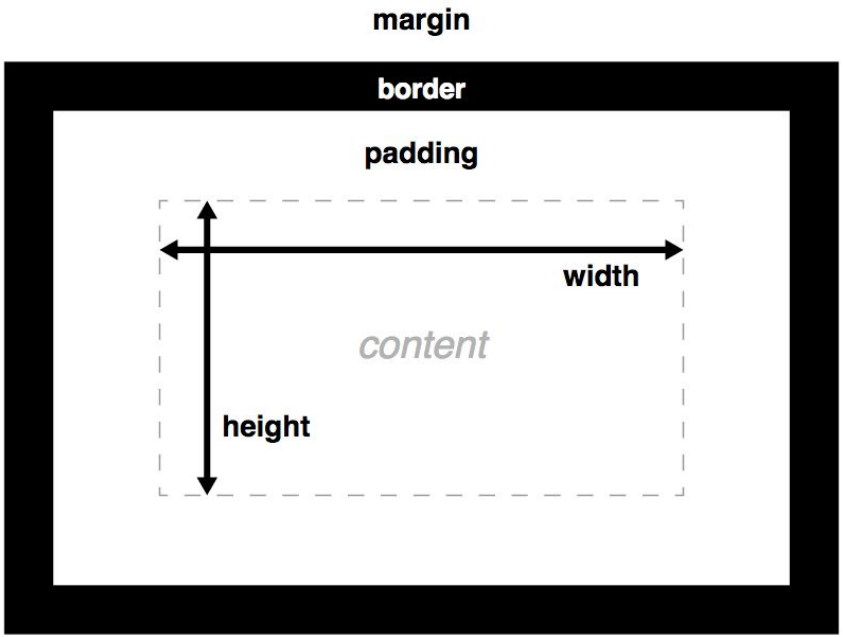
2 способа: через внутреннюю переменную и через параметры

Единицы числовых значений CSS

ИТМО

- Абсолютные:
  - px
  - cm/mm/in (НЕ равны общепринятым единицам)
- Относительные:
  - проценты (%) - процент относительно размера родителя
  - vh/vw/vmax/vmin - относительно высоты/ширины/максимума из ширины и высоты/минимума из ширины и высоты
  - em/rem - пропорции относительно текущего шрифта/шрифта <html> элемента

Box model



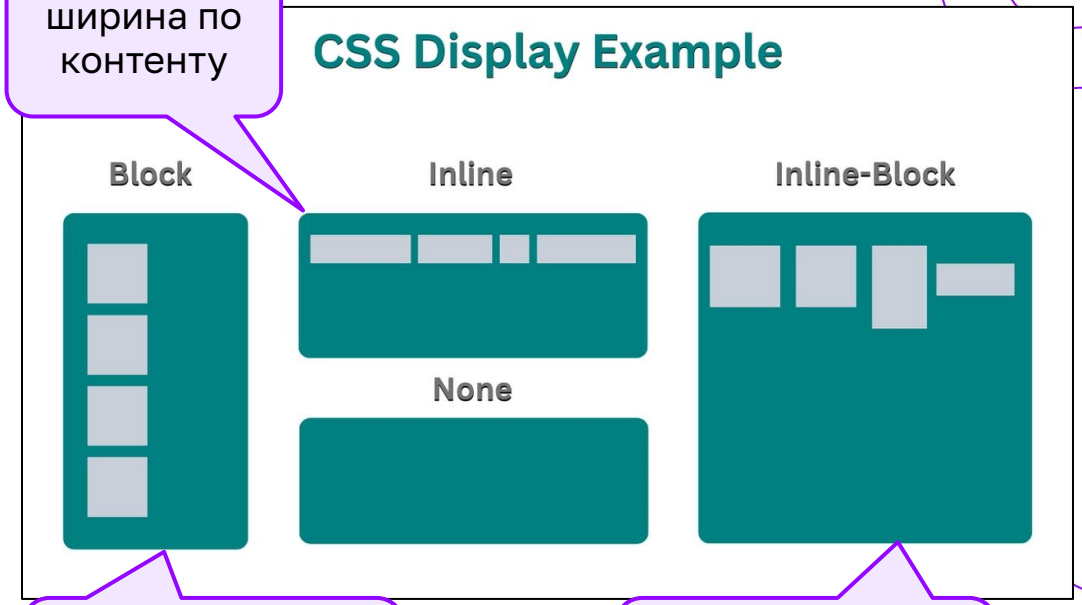
Блочная модель (Box Model) — это набор правил, по которым рассчитывается размер каждого элемента от центра к краям

Задаются эти правила набором css-свойств:

- width - ширина
- height - высота
- padding - внут. отступ
- border - граница
- margin - внеш. отступ

CSS display

ИТМО



ширина по контенту

вся доступная ширина, перенос строки

размеры можно задавать

У каждого элемента есть 2 типа отображения на странице: внешний и внутренний

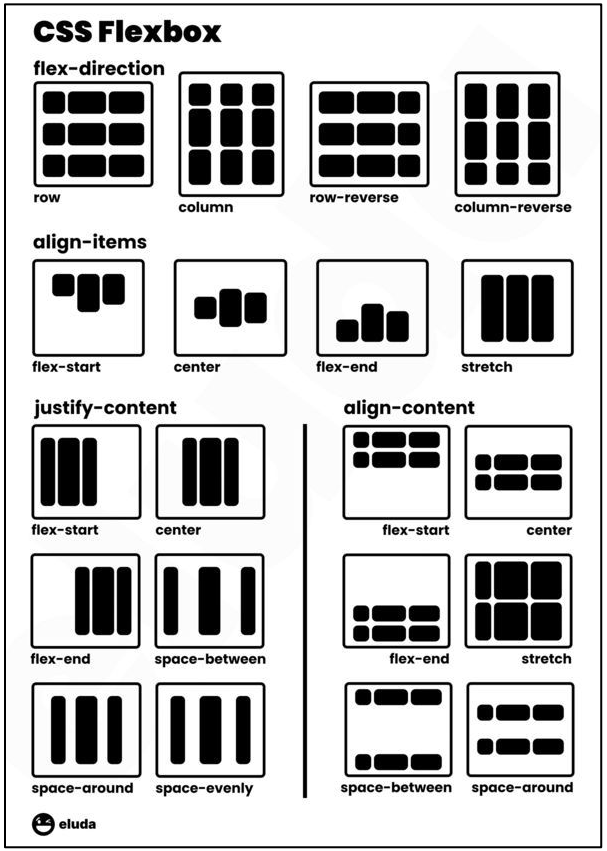
Внутренний определяет отображение дочерних элементов

Внешний определяет расположение элемента в схеме потока

Способ отображения определяется через свойство **display**

CSS Flexbox

ИТМО




```
<meta name="viewport" content="width=device-width, initial
    -scale=1.0">
<title>Форма с Flex</title>
<style>
    body {
        padding: 20px;
    }
    form {
        max-width: 400px;
        margin: 0 auto;
    }
    div {
        display: flex;
        align-items: center;
        margin-bottom: 15px;
    }
    label {
        flex-grow: 0;
        flex-shrink: 0;
        flex-basis: 100px;
        margin-right: 10px;
    }
    input {
        flex-grow: 1;
        flex-shrink: 1;
        flex-basis: auto;
        padding: 8px;
        border: 1px solid #ccc;
        border-radius: 4px;
    }
    button {
        padding: 10px 20px;
        border: 1px solid #ccc;
        border-radius: 4px;
        background: #f0f0f0;
        cursor: pointer;
    }
    .button-wrapper {
        display: flex;
        justify-content: flex-end;
        align-items: center;
    }
</style>
```

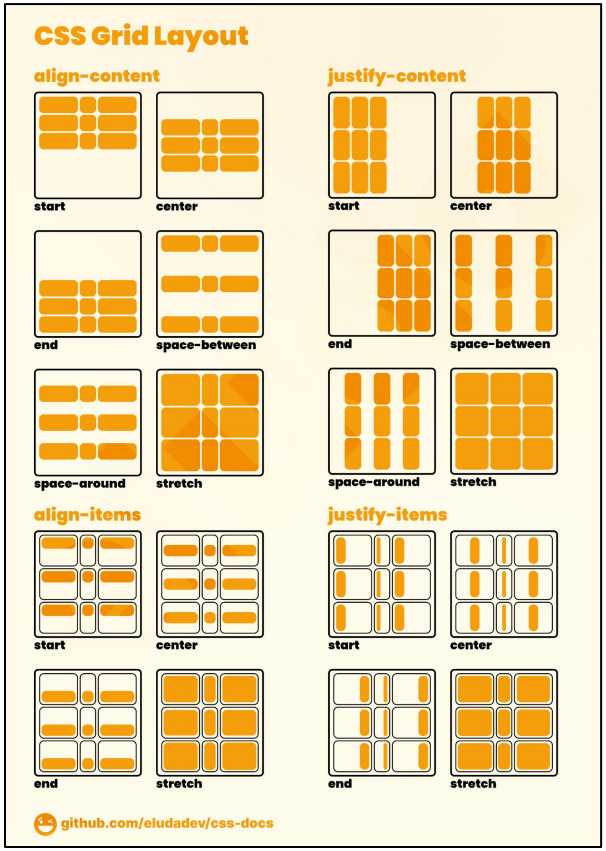
Имя

Email

Пароль

Отправить

CSS Grid




```html
<!DOCTYPE html>
<html lang="ru">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial
        -scale=1.0">
    <title>Форма с Grid</title>
    <style>
        body {
            padding: 20px;
        }
        form {
            max-width: 400px;
            margin: 0 auto;
            display: grid;
            grid-template-columns: 100px 1fr;
            gap: 15px 10px;
            align-items: center;
        }
        label {
            grid-column: 1 / 2;
            justify-self: start;
        }
        input {
            grid-column: 2 / 3;
            padding: 8px;
            border: 1px solid #ccc;
            border-radius: 4px;
        }
        .button-wrapper {
            grid-column: 2 / 3;
            justify-self: end;
        }
        button {
            padding: 10px 20px;
            border: 1px solid #ccc;
            border-radius: 4px;
            background: #f0f0f0;
            cursor: pointer;
        }
    </style>
```

Имя

Email

Пароль

Отправить

CSS Table

ИТМО

```
            border: 1px solid #ccc;
            border-radius: 4px;
            background: #f0f0f0;
            cursor: pointer;
        }
        .button-cell {
            text-align: right;
        }
    </style>
</head>
<body>
    <form action="/submit" method="post">
        <table>
            <tr>
                <td><label for="name">Имя</label></td>
                <td><input type="text" id="name" name="name"></td>
            </tr>
            <tr>
                <td><label for="email">Email</label></td>
                <td><input type="email" id="email" name="email"
                    ></td>
            </tr>
            <tr>
                <td><label for="password">Пароль</label></td>
                <td><input type="password" id="password" name
                    ="password"></td>
            </tr>
            <tr>
                <td></td>
                <td class="button-cell"><button type="submit"
                    >Отправить</button></td>
            </tr>
        </table>
    </form>
</body>
</html>
```



Какую выбрать?

ИТМО

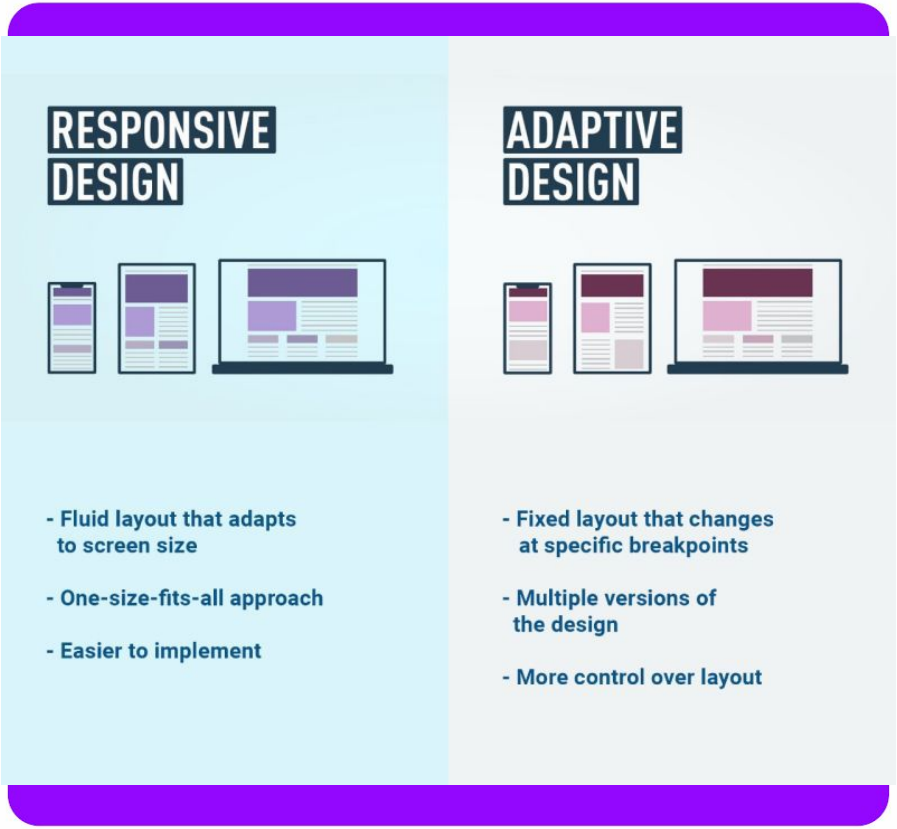


Адаптивный/Отзывчивый дизайн

ИТМО

Адаптивный (Adaptive) дизайн - концепция создания страницы, поддерживающей различные разрешения мониторов для различных устройств

Отзывчивый (Responsive) дизайн - концепция создания страницы с динамической верстkой, меняющейся плавно в зависимости от изменения доступного размера



Адаптивный/Отзывчивый дизайн

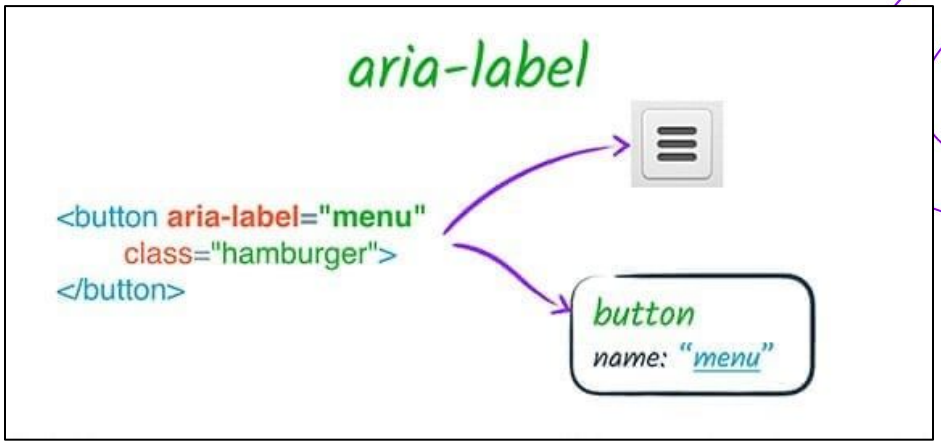
ИТМО

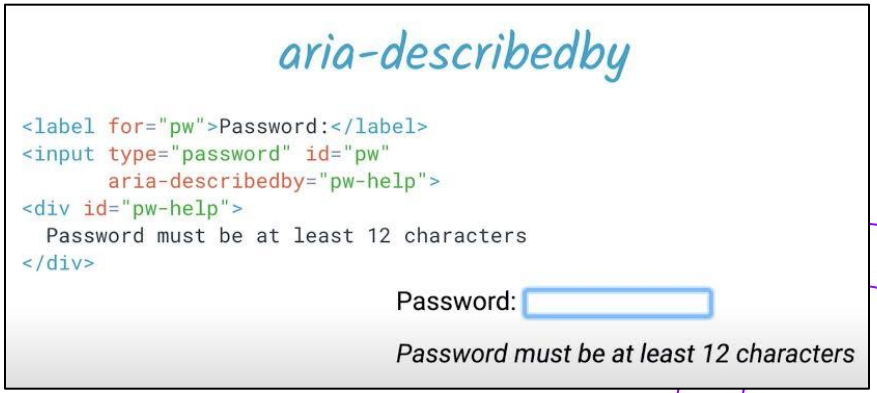
```
@media (max-width: 768px) {
/* Стили для планшетов и
мобильных */
      .container {
        flex-direction: column;
      }
    }
```

1. Относительные единицы: вместо px везде, где можно, используем %, vw, vh, rem
2. Ограничители: для контента часто задают max-width
3. Адаптивные изображения: img { max-width: 100%; }
4. Media Queries: пишем блоки CSS, которые включаются только при определенных условиях.

Доступность или A11y

ИТМО





Доступность - это практика делать сайты удобными в использовании для людей с ограниченными возможностями

- Семантический HTML и семантические элементы
- Иерархия заголовков
- Управление с клавиатуры
- Подписи везде, где нет текста
- Контрастность
- Использования связки Цвет + Описание

JavaScript

Что это

JavaScript - интерпретируемый* ЯП, используемый в веб-разработке для реализации скриптов страниц.

*Если не брать в расчет JIT-компиляцию

Тоже что это

- Придуман в 1995 году Брэнданом Эйхом
- Ядро - ECMAScript (как стандарт, так и базовый язык)
- Актуальная мажорная версия - ECMAScript 16

Синтаксис JS

Уже не актуально, но все еще важно

ИТМО

```
let x = 10;
const name = "John";
```

Способы объявления переменной

```
let x = 10; const name
= "John";
```

Отступы ничего не значат

```
let x = 10 //ошибка
const name = "John";
```

Разделитель в конце

```
if (...) {...}
while (...) {...}
```

Фигурные скобки

```
//однострочный
/*многострочный*/
```

Комментарии

```
import {n} from './f.js';
import m from './f.js';
import * as all from
'./f.js';
```

Импорт зависимостей

Синтаксис JS

ИТМО

Объявления переменных

- const — для данных, которые мы не планируем переназначать
- let — для данных, которые будут изменяться (счетчики, флаги).
- var — как let, но другая область видимости

Примитивные типы данных



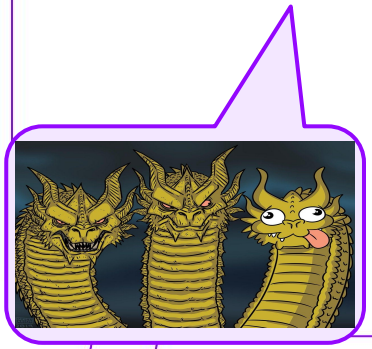
- string — строка текста: "Привет, мир!"
- number — число: 42, 3.14
  Специальные значения: Infinity, NaN (Not a Number)
- boolean — истина или ложь: true/false
- null — намеренное отсутствие значения, «Явно пусто».
- undefined — значение еще не было присвоено, «Неизвестно».

Синтаксис JS

ИТМО

Объявления переменных

- const — для данных, которые мы не планируем переназначать
- let — для данных, которые будут изменяться (счетчики, флаги).
- var — как let, но другая область видимости



Примитивные типы данных

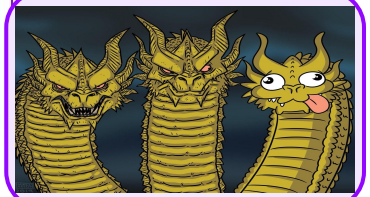
- string — строка текста: "Привет, мир!"
- number — число: 42, 3.14
  Специальные значения: Infinity, NaN (Not a Number)
- boolean — истина или ложь: true/false
- null — намеренное отсутствие значения, «Явно пусто».
- undefined — значение еще не было присвоено, «Неизвестно».

Синтаксис JS

ИТМО

Объявления переменных

- const — для данных, которые мы не планируем переназначать
- let — для данных, которые будут изменяться (счетчики, флаги).
- var — как let, но другая область видимости



Разная область видимости!

Примитивные типы данных

- string — строка текста: "Привет, мир!"
- number — число: 42, 3.14
  Специальные значения: Infinity, NaN (Not a Number)
- boolean — истина или ложь: true/false
- null — намеренное отсутствие значения, «Явно пусто».
- undefined — значение еще не было присвоено, «Неизвестно».

Операторы, условия, циклы

ИТМО

```
let sum = 1 + 1 //сложение
let subtr = 1 - 1
//вычитание
let multipl = 1 * 2
//умножение
let div = 1 / 1 //деление
let mod = 10 % 2 //mod
```

Арифметические операторы

```
Сравнения: >, <, == (с
приведением типов),
=== (без приведения
типов)
Логические: && (И), ||
(ИЛИ), ! (НЕ)
```

Операторы сравнения/логические

```
if (temperature > 25) {
  console.log('Жарко!');
} else if (temperature <
10) {
  console.log('Холодно!');
} else
console.log('Нормально');
```

Условные конструкции

```
let result = age >= 18
? 'Взрослый' :
'Ребенок';
```

Тернарный оператор

```
switch (type) {
  case (‘Обед’):
     console.log('Хорошо')
;
     break;
  default:
     console.log('Плохо');
}
```

switch case

```
for (let i = 0; i < 5;
i++) {...
for (const item [of/in]
array) {...

while (condition) {...
do {...} while (condition)
```

Цикл for/while

Функции

ИТМО

```
function sum(a, b) {
      return a + b;
}
```

Function Declaration

Объявляется ключевым словом function

Всплывает (hoisting) — можно вызвать ДО объявления в коде

```
const multiply =
function(a, b) {
      return a * b;
};
```

Function Expression

Функция присваивается переменной

Не всплывает — можно вызвать только ПОСЛЕ объявления

```
const subtract = (a, b) =>
a - b;
```

Arrow Function

Короткий синтаксис, не имеет arguments

Идеальна для колбэков и функций без своего контекста

Области видимости

```
let global = "Я виден всем";

function testScope() {
    let functional = "Я только внутри функции";
    var oldVar = "Я тоже только внутри функции (var)";

    if (true) {
        let block = "Я только внутри if";
        var notBlock = "Я утекаю из if";

        console.log(global);        // Я виден всем
        console.log(functional);    // Я только внутри функции
        console.log(block);         // Я только внутри if
        console.log(oldVar);        // Я тоже только внутри функции (var)
    }

    console.log(notBlock);          // Я утекаю из if
    console.log(oldVar);            // Я тоже только внутри функции (var)
    console.log(block);             // ReferenceError
}

testScope();

console.log(global);                // Я виден всем
console.log(functional);            // ReferenceError
console.log(notBlock);              // ReferenceError
console.log(oldVar);                // ReferenceError
```

Hoisting (поднятие) - механизм перемещения объявленных конструкций в самый верх их области видимости

Применяется к:

- функциям различных объявлений
- переменным

IIFE

ИТМО

```
(function print(output) {
     console.log(output);
})();
```

IIFE (немедленно вызываемое функциональное выражение) — это функция, которая объявляется и вызывается немедленно в одном выражении. Она выполняется ровно один раз сразу после создания.

this JS

ИТМО

this - ключевое слово, возвращающее объект текущей области видимости объекта, в рамках которого мы выполняем код.

Определяется в момент вызова и зависит от места вызова. Глобальная область видимости - тоже область видимости объекта. В браузере - реализация интерфейса Window.

В методе this - область видимости объекта. Функции и блоки кода при вызове this отсылают на область видимости объекта, который их содержит, в момент их исполнения

Что это такое и почему так

ИТМО

```
function fn() {
    let count = 1;
    function increase() {
        count++;
        console.log(count);
    }
    return increase;
}

const fnExpr = fn();

fnExpr();
fnExpr();
fnExpr();
```

```
2
3
4

=== Code Execution Successful ===
```

Замыкание

ИТМО

```
function fn() {
    let count = 1;
    function increase() {
        count++;
        console.log(count);
    }
    return increase;
}

const fnExpr = fn();

fnExpr();
fnExpr();
fnExpr();
```

```
2
3
4

=== Code Execution Successful ===
```

increase сохраняет доступ к count из ЛО fn

1 вызов: 1+1=2
2 вызов: 2+1=3
3 вызов: 3+1=4

Замыкание — это функция, которая запоминает свое лексическое окружение. Выполнение такой функции будет производится в контексте ее окружения. Каждая функция в JS - замыкание

Лексическое окружение - внутренняя структура данных, хранящая информацию о переменных и структурах кода. У каждой функции лексическое окружение уникально

snapback to Функции

ИТМО

Контекст запоминается в момент создания!

```
function sum(a, b) {
      return a + b;
}
```

Function Declaration

Объявляется ключевым словом function

Всплывает (hoisting) — можно вызвать ДО объявления в коде

```
const multiply =
function(a, b) {
      return a * b;
};
```

Function Expression

Функция присваивается переменной

Не всплывает — можно вызвать только ПОСЛЕ объявления

```
const subtract = (a, b) =>
a - b;
```

Arrow Function

Короткий синтаксис, **не имеет своего this**, не имеет arguments

Идеальна для колбэков и функций без своего контекста

Массивы

ИТМО

```
const goods = ['Хлеб', 'Молоко'];
console.log(todos[0]); // 'Хлеб'

const filteredGoods = goods.filter(good => good == "Хлеб")
// ['Хлеб']

const mappedGoods = goods.map(good => good[0])
// ['Х', 'М']

const mappedGoods = goods.find(good => good == "Хлеб")
// Хлеб

goods.forEach(good => console.log)
// Хлеб
// Молоко
```



Массив — упорядоченный список чего угодно. Элементы получаем по индексу, начиная с 0

Методы

- map — преобразует каждый элемент, создавая новый массив
- filter — оставляет только те элементы, что проходят проверку
- find — находит первый подходящий элемент
- forEach — просто выполняет действие для каждого элемента

Объекты

ИТМО

```
const user = {
    name: 'Иван',
    age: 20,
    greet() {
        console.log(`Привет, я ${this.name}`);
    }
};

class Person {
  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  greet() {
    console.log(`Привет, я ${this.name}`);
  }
}

const ivan = new Person('Иван', 20);
```

Объект — это «словарь». Хранит пары «ключ-значение»

Свойства можно получать через точку (`user.name`) или через квадратные скобки (`user['name']`).

Является ссылочным типом данных

Деструктуризация

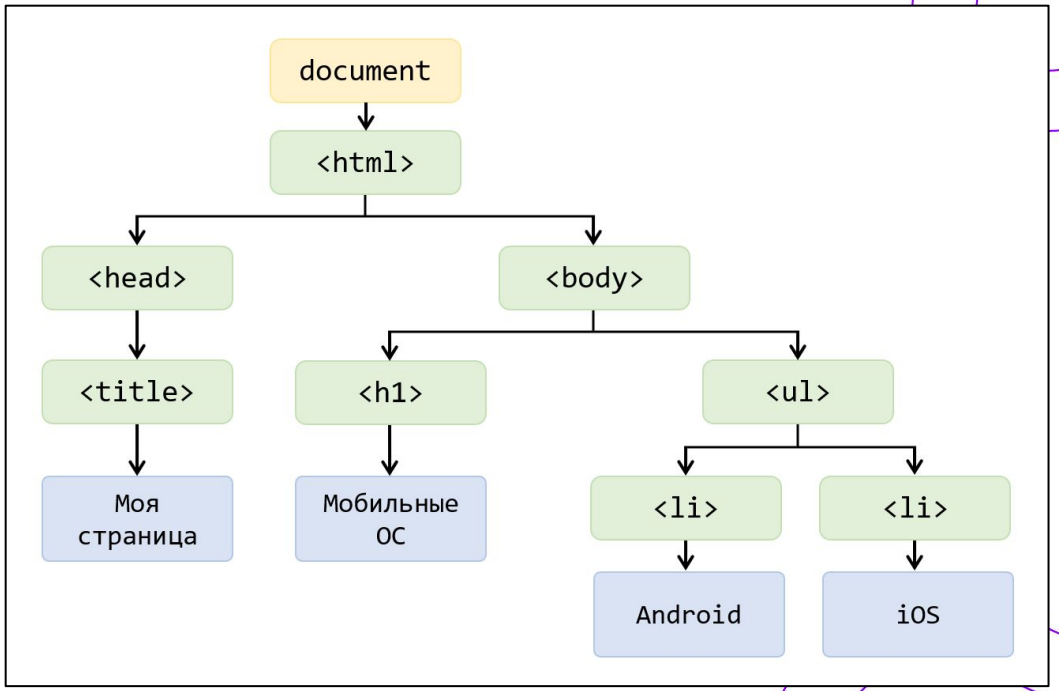
ИТМО

```
const { name, age } = user;
const [firstTodo, secondTodo]
= todos;
```

Деструктуризация - синтаксис, который позволяет «распаковывать» массивы и объекты

DOM

ИТМО



Document Object Model — это объектная модель документа

Доступ через объект document

Основные методы поиска:

- document.querySelector
- document.querySelectorAll
- document.getElementById

DOM

ИТМО

Что еще?

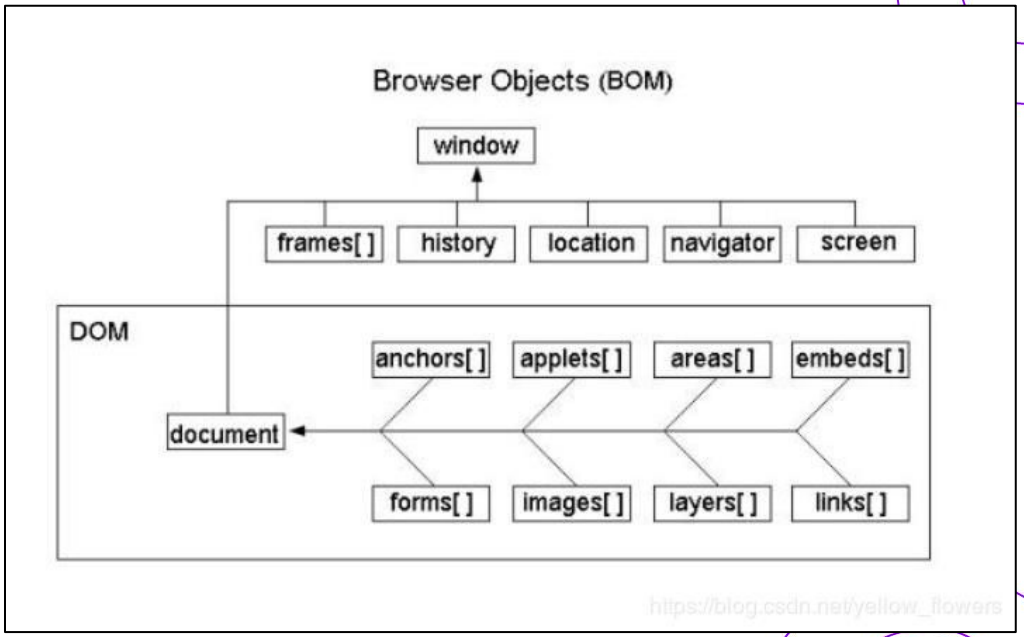
- Прочитать или изменить текст: element.textContent (безопасно) или element.innerHTML (небезопасно!)
- Управлять классами: element.classList.add('active'), classList.remove(), classList.toggle()
- Изменять атрибуты: element.setAttribute('src', 'new-image.jpg')
- Многое другое...



```
// Создали элемент
const newParagraph = document.createElement('p');
// Заполнили текстом
newParagraph.textContent = 'Новый абзац';
// Добавили в конец body
document.body.append(newParagraph);
```

BOM

ИТМО

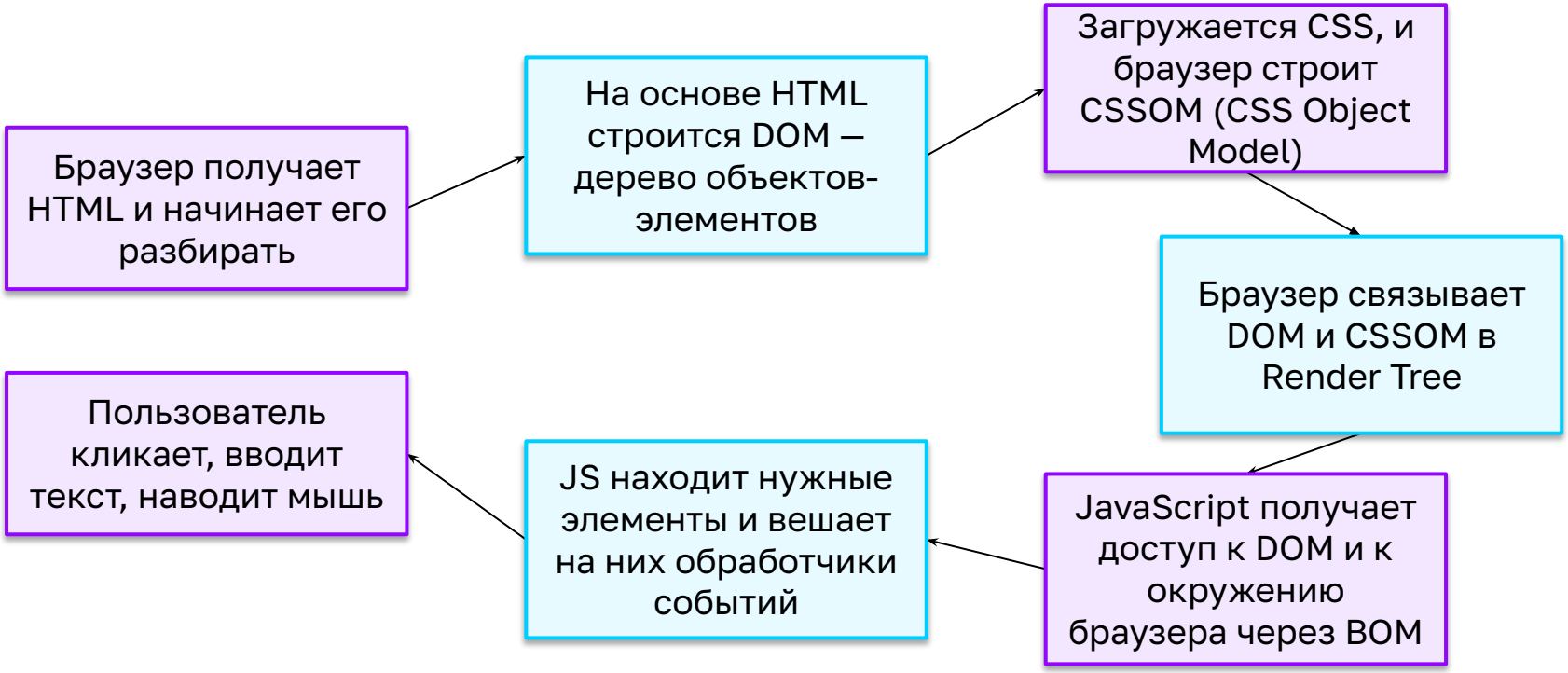


Browser Object Model — это объектная модель браузера, доступ через объект window (хотя можно и без него)

- window: alert(), confirm(), prompt(), innerWidth, innerHeight — размеры окна
- location: Позволяет прочитать текущий URL или принудительно перейти на другой
- history: доступ к истории переходов, вернуться назад, менять URL без перезагрузки
- navigator: Информация о браузере (язык пользователя, есть ли интернет)
- Хранилища: localStorage (живет вечно) и sessionStorage (живет до закрытия вкладки)
- Таймеры: setTimeout(func, 2000), setInterval(func, 1000)

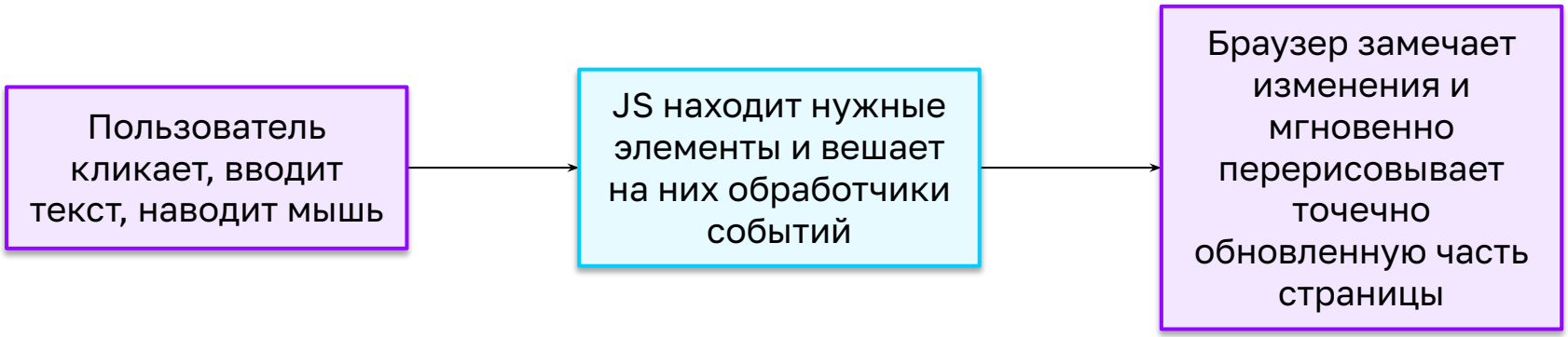
Как это все работает вместе №1

ИТМО



Как это все работает вместе №2

ИТМО



Спасибо
за внимание!

ITMO more than a UNIVERSITY